Operating Systems

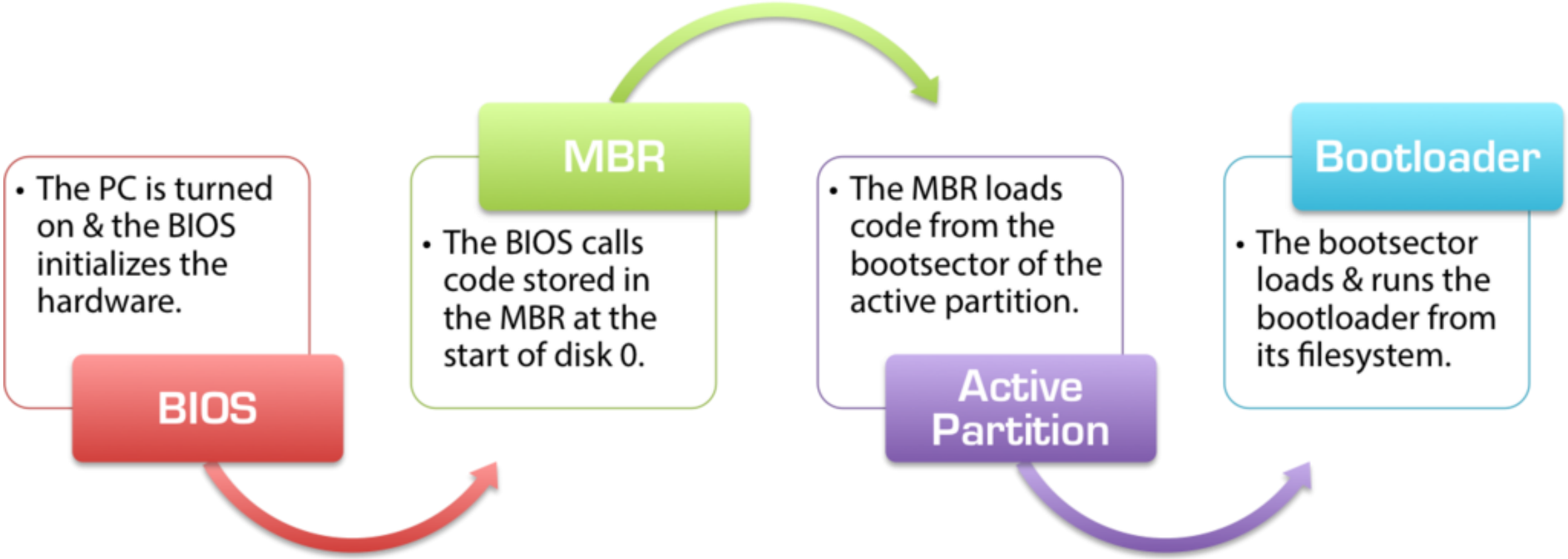
Dmitry Zaitsev

Lecture 16: Booting OS from different media. Case study: configuring Grub in Linux.

OS booting history

- Booting comes from “pulling yourself up by your bootstraps” — meaning a small program loads a larger program (the OS).
- Early computers (1950s) – manual booting, switches, punched cards
- Batch systems (1960s) – a small program called a bootstrap loader was stored in read-only memory.
- IBM PC (1970s) – BIOS and Disk Booting: BIOS finds bootable device and loads the boot sector from a disk
- Bootloaders and Multi-OS Systems (1980s–1990s): LILO, GRUB, NTLDR – boot menu, recovery
- UEFI () Replaces BIOS (2000s) – faster, large disk, GPT instead of MBR,
- Modern – secure boot, TPM, encrypted boot drives

BIOS/MBR Boot Process



MBR structure

MBR – first sector (LBA 0 or CHS 0,0,1) of a bootable disk

CC BY

MASTER BOOT RECORD

≥ INVOKE-IR

BY: JARED ATKINSON
TEMPLATE BY: ANGE ALBERTINI

```
000: 33 C0 8E D0 BC 00 7C 8E C0 8E D8 BE 00 7C BF 00
010: 06 B9 00 02 FC F3 A4 50 68 1C 06 CB FB B9 04 00
020: BD BE 07 80 7E 00 00 7C 0B 0F 85 0E 01 83 C5 10
030: E2 F1 CD 18 88 56 00 55 C6 46 11 05 C6 46 10 00
040: B4 41 BB AA 55 CD 13 5D 72 0F 81 FB 55 AA 75 09
050: F7 C1 01 00 74 03 FE 46 10 66 60 80 7E 10 00 74
060: 26 66 68 00 00 00 00 66 FF 76 08 68 00 00 68 00
070: 7C 68 01 00 68 10 00 B4 42 8A 56 00 8B F4 CD 13
080: 9F 83 C4 10 9E EB 14 B8 01 02 BB 00 7C 8A 56 00
090: 8A 76 01 8A 4E 02 8A 6E 03 CD 13 66 61 73 1C FE
0A0: 4E 11 75 0C 80 7E 00 80 0F 84 8A 00 B2 80 EB 84
0B0: 55 32 E4 8A 56 00 CD 13 5D EB 9E 81 3E FE 7D 55
0C0: AA 75 6E FF 76 00 E8 8D 00 75 17 FA B0 D1 E6 64
0D0: E8 83 00 B0 DF E6 60 E8 7C 00 B0 FF E6 64 E8 75
0E0: 00 FB B8 00 BB CD 1A 66 23 C0 75 3B 66 81 FB 54
0F0: 43 50 41 75 32 81 F9 02 01 72 2C 66 68 07 BB 00
100: 00 66 68 00 02 00 00 66 68 08 00 00 00 66 53 66
110: 53 66 55 66 68 00 00 00 00 66 68 00 7C 00 00 66
120: 61 68 00 00 07 CD 1A 5A 32 F6 EA 00 7C 00 00 CD
130: 18 A0 B7 07 EB 08 A0 B6 07 EB 03 A0 B5 07 32 E4
140: 05 00 07 8B F0 AC 3C 00 74 09 BB 07 00 B4 0E CD
150: 10 EB F2 F4 EB FD 2B C9 E4 64 EB 00 24 02 E0 F8
160: 24 02 C3 49 6E 76 61 6C 69 64 20 70 61 72 74 69
170: 74 69 6F 6E 20 74 61 62 6C 65 00 45 72 72 6F 72
180: 20 6C 6F 61 64 69 6E 67 20 6F 70 65 72 61 74 69
190: 6E 67 20 73 79 73 74 65 6D 00 4D 69 73 73 69 6E
1A0: 67 20 6F 70 65 72 61 74 69 6E 67 20 73 79 73 74
1B0: 65 6D 00 00 00 63 7B 9A 82 D4 BA 7D 00 00 00 20
1C0: 21 00 07 FE FF FF 00 08 00 00 00 90 36 06 80 FE
1D0: FF FF 07 FE FF FF 00 A0 36 06 00 60 09 00 00 00
1E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
1F0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 55 AA
```

	FIELDS	VALUES
BOOT CODE	jump to boot program	
	disk parameters	
	boot program code	
	disk signature	82D4BA7D
PARTITION TABLE	status	0x00 - Non-Bootable
	starting head	0x20
	starting sector	0x21
	starting cylinder	0x00
	partition type	0x07 - NTFS
	ending head	0xFE
	ending sector	0x3F
	ending cylinder	0x3FF
	relative start sector	0x800
	total sectors	0x6369000
	status	0x80 - Bootable
	starting head	0xFE
	starting sector	0x3F
	starting cylinder	0x3FF
	partition type	0x07 - NTFS
	ending head	0xFE
	ending sector	0x3F
	ending cylinder	0x3FF
	relative start sector	0x636A000
	total sectors	0x96000
	partition type	0x00 - EMPTY
	partition type	0x00 - EMPTY
END OF MBR	marker	0x55AA

CHS ADDRESSING

```
00100000 00100001 00000000
00100000 100001 0000000000
Head     - 1st byte
Sector   - 2nd byte (0-5 bits)
Cylinder - 2nd byte (6-7 bits)
           3rd byte
```

PARTITION TYPES

0x00 - EMPTY	0x83 - LINUX
0x01 - FAT12	0x84 - HIBERNATION
0x04 - FAT16	0x85 - LINUX_EXTENDED
0x05 - MS_EXTENDED	0x86 - NTFS_VOLUME_SET
0x06 - FAT16	0x87 - NTFS_VOLUME_SET_1
0x07 - NTFS	0xa0 - HIBERNATION_1
0x0b - FAT32	0xa1 - HIBERNATION_2
0x0c - FAT32	0xa5 - FREEBSD
0x0e - FAT16	0xa6 - OPENBSD
0x0f - MS_EXTENDED	0xa8 - MACOSX
0x11 - HIDDEN_FAT12	0xa9 - NETBSD
0x14 - HIDDEN_FAT16	0xab - MAC_OSX_BOOT
0x16 - HIDDEN_FAT16	0xb7 - BSDI
0x1b - HIDDEN_FAT32	0xb8 - BSDI_SWAP
0x1c - HIDDEN_FAT32	0xee - EFI_GPT_DISK
0x1e - HIDDEN_FAT16	0xef - EFI_SYSTEM_PARTITION
0x42 - MS_MBR_DYNAMIC	0xfb - VMWARE_FILE_SYSTEM
0x82 - SOLARIS_X86	0xfc - VMWARE_SWAP
0x82 - LINUX_SWAP	

MBR boot details

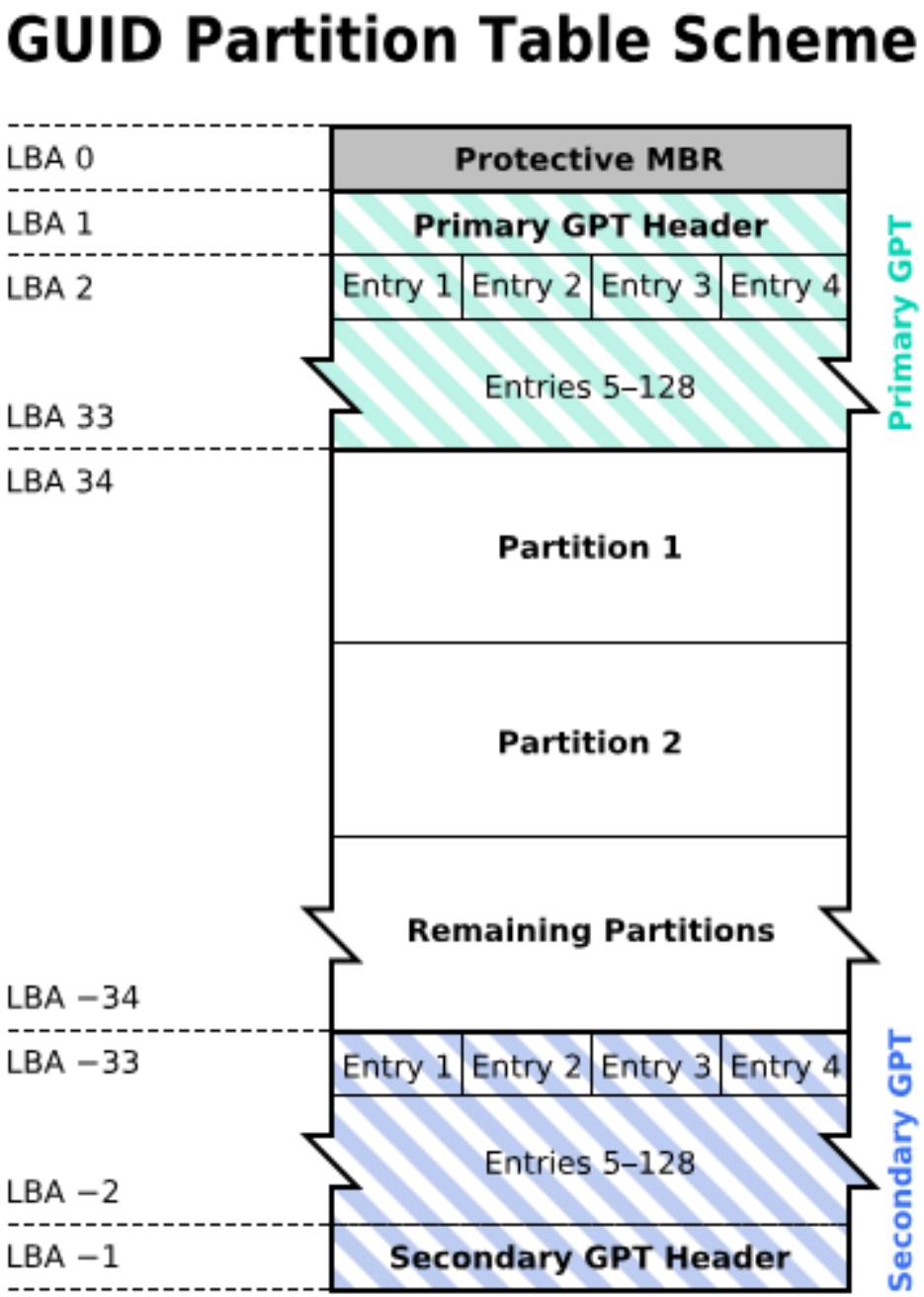
- **Power On: Computer starts.**
- **BIOS Runs and Performs POST (Power-On Self-Test): Searches for bootable device (HDD, USB, CD).**
- **BIOS Loads MBR: BIOS loads the first 512 bytes of disk into RAM. Execution jumps to MBR bootloader code.**
- **MBR Finds Active Partition: Looks at partition table. Finds the active (bootable) partition.**
- **Loads Boot Sector of Active Partition: Loads Volume Boot Record (VBR). VBR loads OS loader (NTLDR, BOOTMGR).**
- **OS Kernel Loads: Operating system starts. Login screen appears.**

BIOS vs UEFI

BIOS
Basic Input Output System
MBR
Master Boot Record
Boot Loader
Kernel
Operating System

UEFI
Unified Extensible Firmware Interface
EFI Boot Loader
Kernel
Operating System

GUID Partition Table (GPT)
Globally Unique Identifier



A **GUID** is a 128-bit number used to uniquely identify something (like a partition, file, user, or software component).
It ensures that the identifier is **unique across the world**.

MBR vs GPT

Limitations of MBR:

- Supports only up to 2 TB disk size
- Only 4 primary partitions
- No backup of partition table
- Less secure than GPT

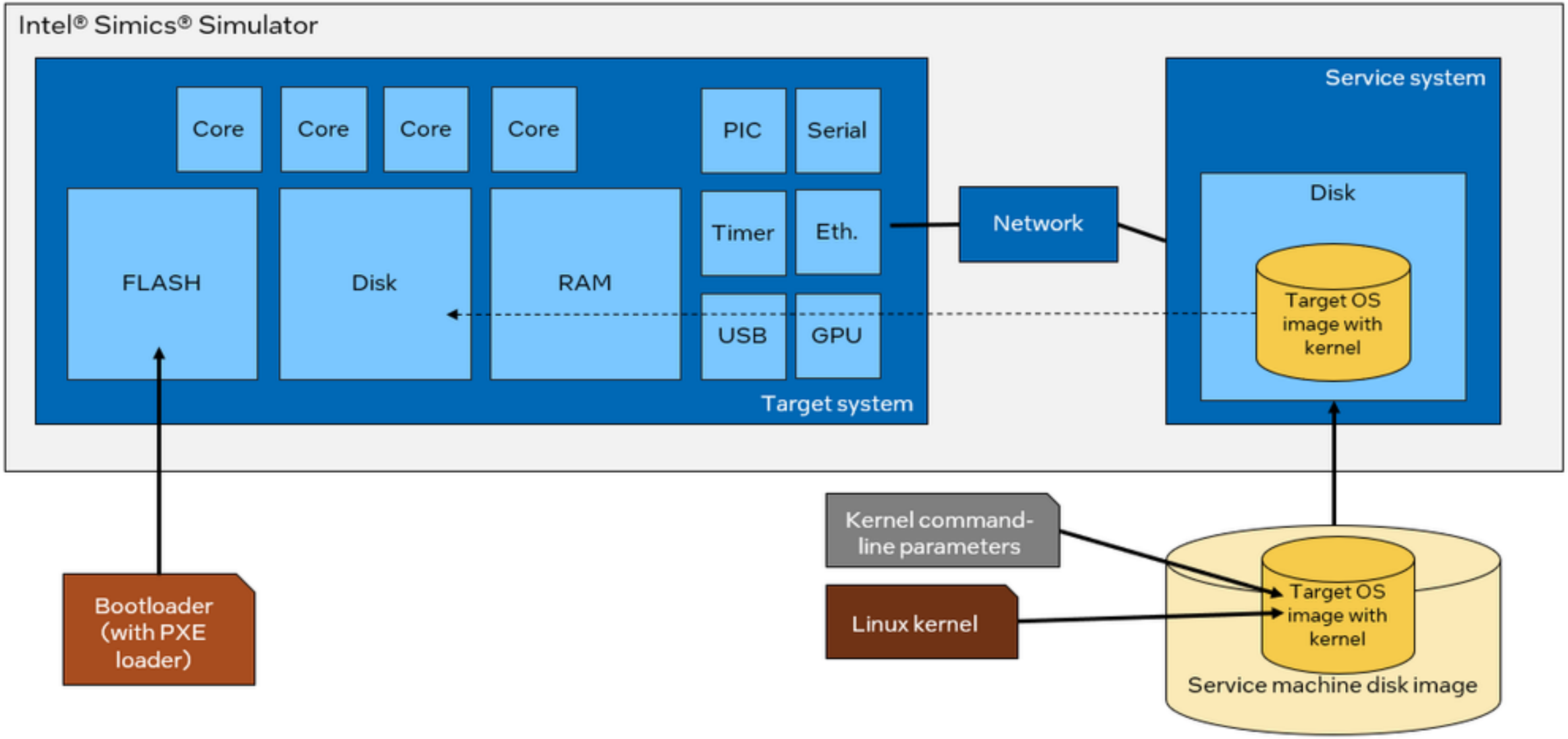
Advantages of GPT:

- Can handle disks up to 9.4 Zettabytes
- Supports up to 128 partitions
- Backups Partition Table at the end of disk
- Improved Data Integrity
- Enables secure boot and reduces boot time
- GUID Prevents partition conflicts

OS start-up process

- OS Kernel Loads. Kernel initializes CPU structures – interrupts and control registers; memory management – paging and virtual memory; starts device drivers; mounts file systems
- Optional: mounts initial RAM disk to access other components beyond the kernel such as loadable modules and basic services
- System Services Start. Background services and system processes start: Networking, Security services, System logging
- User Interface Loads. Command Line Interface (CLI) and/or Graphical User Interface (GUI)

Booting from network



Linux start-up process

- GRUB EFI file is loaded and run
- GRUB displays boot menu, finds Linux kernel image (vmlinuz) and loads it into memory, loads temporary RAM file system (initramfs or initrd), handles control to kernel
- Kernel initialization process: CPU & memory initialization; essential device drivers loaded from initramfs; mounts root file system; starts systemd (PID 1)
- systemd starts system services: networking networkd; logging journald; security farewalld; device services; handles targets
- User Interface Startup: graphical.target GUI - GNOME; multi-user.target LCI - getty

GRUB – GRand Unified Bootloader

- What is GRUB?
  - GRUB is a bootloader used in Linux and other Unix-like systems.
  - Its main job: load the OS kernel into memory and start the operating system.
- Key Features of GRUB:
  - Multi-boot support – can choose between multiple OSes.
  - Filesystem support – can read partitions formatted as ext4, NTFS, FAT32, etc.
  - Command-line interface – allows manual booting and kernel parameter editing.
  - Works with MBR and GPT – compatible with BIOS and UEFI systems.

GRUB configuration files

- Which operating systems to display in the boot menu. Where kernels are located. Boot parameters and options.
- /etc/default/grub – main configuration file: GRUB_TIMEOUT, GRUB_DEFAULT, GRUB_CMDLINE_LINUX
- /boot/grub/grub.cfg – bootloader configuration file read by GRUB at startup: menu entries for all OSes and kernels; commands to load kernels and initramfs; chainloading commands for other Oses
- /etc/grub.d/ – scripts that generate parts of grub.cfg
- sudo update-grub or sudo grub-mkconfig -o /boot/grub/grub.cfg

Example of /etc/default/grub

```
GRUB_DEFAULT=0
GRUB_TIMEOUT_STYLE=hidden
GRUB_TIMEOUT=0
GRUB_DISTRIBUTOR=`( . /etc/os-release; echo ${NAME:-Ubuntu} ) 2>/dev/null || echo Ubuntu`
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
GRUB_CMDLINE_LINUX=""
```

Example of /boot/grub/grub.cfg

```
menuentry 'Debian GNU/Linux, with Linux 6.12.8-amd64' --class debian
--class gnu-linux --class gnu --class os $menuentry_id_option
'gnulinux-6.12.8-amd64-advanced-5522bbcf-dc03-4d36-a3fe-2902be93
8ed4' { …
    insmod gzio
    insmod part_gpt
    insmod ext2
    set root='hd0,gpt2'
    linux /vmlinuz-6.12.8-amd64
root=UUID=fa12b345-cd67-89ef-0123-456789abcdef ro quiet splash
    initrd /initramfs-6.12.8-amd64.img
}
```

How systemd starts up the system

- Activate all units that are dependencies of default.target (as well as recursively all dependencies of dependencies).
- Usually, default.target is simply an alias of graphical.target or multi-user.target, depending on whether the system is configured for a graphical UI or only for a text console.
- To enforce minimal ordering between the units pulled in, a number of well-known target units are available, as listed on systemd.special.

Configuring systemd

- changing boot targets
- managing services
- adjusting startup behavior

Switching target

- systemctl list-units --type target
- systemctl isolate multi-user.target
- systemctl isolate graphical.target

Switching default target

- systemctl get-default
- sudo systemctl set-default multi-user.target
- sudo systemctl reboot
- sudo systemctl set-default graphical.target
- sudo reboot

Start/stop service

- systemctl list-unit-files --type service –all
- systemctl status apache2
- sudo systemctl enable apache2
- sudo systemctl disable apache2

systemd.unit - Unit configuration

A unit file is a plain text ini-style file that encodes information about

- a service, a socket, a device, a mount point, an automount point, a swap file or partition,
- a start-up target, a watched file system path, a timer controlled and supervised by systemd,
- a resource management slice or a group of externally created processes.

Unit types

Unit Type	Filename Extension	Type Description
Service	.service	System service.
Target	.target	Group of systemd units.
Automount	.automount	File system auto-mount point.
Device	.device	Device file recognized by the kernel.
Mount	.mount	File system mount point.
Path	.path	File or directory in a file system.
Scope	.scope	Externally created process.
Slice	.slice	Group of hierarchically organized units that manage system processes.
Snapshot	.snapshot	Saved state of the systemd manager.
Socket	.socket	Inter-process communication socket.
Swap	.swap	Swap device or a swap file.
Timer	.timer	Systemd timer.

local-fs-pre.target

```
local-fs-pre.target
         |
         v
(various mounts and   (various swap   (various cryptsetup
 fsck services...)     devices...)        devices...)       (various low-level   (various low-level
         |                  |                  |             services: udevd,     API VFS mounts:
         v                  v                  v             tmpfiles, random     mqueue, configfs,
  local-fs.target      swap.target     cryptsetup.target    seed, sysctl, ...)      debugfs, ...)
         |                  |                  |                     |                    |
         \__________________|__________________ | ____________________|____________________/
                                              \|/
                                               v
                                        sysinit.target
```

sysinit.target

```
                                    sysinit.target
                                           |
      ____________________________________/|\____________________________________
     /                  |                  |                    |                  \
     |                  |                  |                    |                  |
     v                  v                  |                    v                  v
  (various           (various              |                (various         rescue.service
 timers...)          paths...)             |               sockets...)              |
     |                  |                  |                    |                  v
     v                  v                  |                    v             rescue.target
timers.target      paths.target            |             sockets.target
     |                  |                  |                    |
     v                  \_________________ | ___________________/
                                          \|/
                                           v
                                       basic.target
```

basic.target

```
                                        basic.target
                                              |
        ____________________________________/|                                emergency.service
       /                  |                  |                                        |
       |                  |                  |                                        v
       v                  v                  v                                emergency.target
    display-        (various system    (various system
manager.service         services           services)
       |              required for            |
       |             graphical UIs)           v
       |                  |           multi-user.target
       |                  |                  |
       \_________________ | _________________/
                         \|/
                          v
                 graphical.target
```

Configure a Custom Service

- [Create a unit file in /etc/systemd/system/]
- Description=My Custom Service
- After=network.target

- [Service]
- Type=simple
- ExecStart=/usr/bin/myprogram
- Restart=on-failure

- [Install]
- WantedBy=multi-user.target